EKS GitOps Platform

Amazon EKS with ArgoCD, AWS Controllers for Kubernetes (ACK), and Kube Resource Orchestrator (kro)

Document type: Technical Implementation Guide
Project: smart-infra-manager-eks
Repository: github.com/CloudScope/eks-argo-ack-kro
Region: ap-south-1
Kubernetes version: 1.36
Prepared by: Platform Engineering
Date: 2026-06-28

Contents

1. Executive Summary

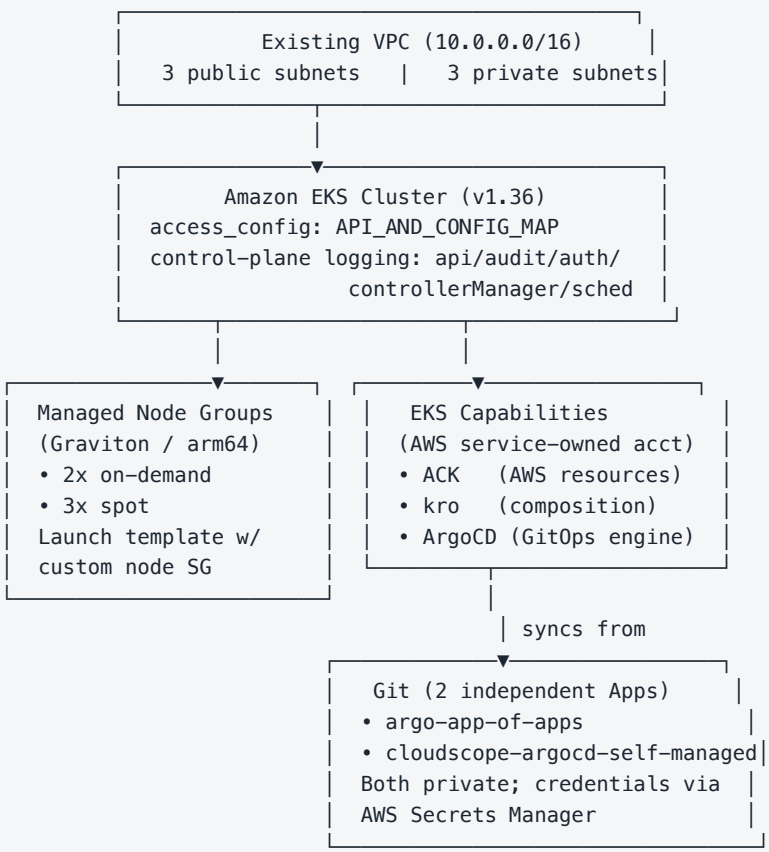
This document describes the design and build-out of a production EKS platform that uses three native **Amazon EKS Capabilities** — ArgoCD, AWS Controllers for Kubernetes (ACK), and Kube Resource Orchestrator (kro) — to move AWS resource provisioning out of Terraform and into a Git-reviewed, continuously-reconciled GitOps workflow.

The platform follows a deliberate two-layer ownership model:

- **Terraform owns the bootstrap/platform layer**: VPC consumption, the EKS cluster, node groups, the three capabilities themselves and their IAM roles, Pod Identity wiring, and the GitOps bootstrap (root `Application` resources and Git credentials).
- **ArgoCD + ACK + kro own the application layer**: AWS resources individual workloads need (S3 buckets, databases, queues, IAM roles, etc.) and higher-level composed templates (kro `ResourceGraphDefinition`s), declared as YAML in Git and reconciled continuously.

This split exists because GitOps tooling cannot manage the infrastructure it itself runs on — the cluster has to exist before ArgoCD can run on it, so cluster lifecycle stays in Terraform.

2. Architecture Overview



Layer	Owns	Managed by
Network	VPC, subnets, NAT, route tables	Terraform `terraform/main.tf` (pre-existing, referenced)
Cluster	EKS control plane, node groups, add-ons	Terraform `eks_cluster.tf, node_groups.tf`
Capabilities	ACK, kro, ArgoCD capability resources + IAM	Terraform `capabilities.tf`
GitOps bootstrap	Root Applications, repo creds, cluster registration	Terraform `argocd_apps.tf`
Application resources	S3/RDS/SQS/IAM objects, kro templates	GitOps `Git repos`
Secrets	GitHub PAT, app secrets	AWS `Secrets Manager` (never in TF state)

3. Prerequisites

- AWS account with an existing VPC (3 public + 3 private subnets across 3 AZs)
- Terraform >= 1.0 (pinned to 1.14.6 in CI), AWS provider >= 6.25.0 (required for the native `aws_eks_capability` resource, introduced in that release)
- AWS CLI v2 available wherever `terraform apply` runs (used by the Kubernetes provider's `exec` auth plugin)
- AWS IAM Identity Center already enabled for the organization (exactly one instance is supported per org; Terraform resolves it automatically)
- A GitHub Personal Access Token with read access to every private repo ArgoCD will sync from, stored directly in AWS Secrets Manager (never in Terraform state or tfvars)
- GitHub Actions configured with an OIDC-federated IAM role for `terraform plan` / `apply`

4. Infrastructure Components (Terraform)

4.1 EKS Cluster

```
resource "aws_eks_cluster" "main" {
  name     = var.cluster_name
  version  = var.cluster_version          # 1.36
  role_arn = aws_iam_role.cluster.arn

  access_config {
    authentication_mode = "API_AND_CONFIG_MAP"   # required for EKS Access Entries
  }

  vpc_config {
    subnet_ids              = local.all_subnet_ids   # public + private
    endpoint_private_access = true
    endpoint_public_access  = true
  }

  enabled_cluster_log_types = ["api","audit","authenticator","controllerManager","scheduler"]
}
```

> **Why API_AND_CONFIG_MAP:** EKS Access Entries (used throughout this platform for capability and GitOps-principal authorization) require API or API_AND_CONFIG_MAP authentication mode. CONFIG_MAP-only clusters cannot use Access Entries at all.

4.2 Node Groups — Graviton, Fixed-Size, Mixed Capacity

Group	Capacity type	Size	Instance types	AMI type
on_demand	ON_DEMAND	min=max=desired=2	t4g.large, m6g.large	AL2023_ARM_64_STANDARD
spot	SPOT	min=max=desired=3	t4g.large, m6g.large	AL2023_ARM_64_STANDARD

Both groups share a single `aws_launch_template` that attaches a custom node security group and a 50 GiB gp3 root volume — `aws_eks_node_group` has no `vpc_config` /security-group argument of its own, so a launch template is the only way to apply a custom SG to managed node group instances.

> **Why Graviton:** up to ~40% better price/performance than equivalent x86_64 instances per AWS's own benchmarks. Both node groups must stay single-architecture — EKS node groups cannot mix arm64 and x86_64 instance types in one group.

> **Why fixed min=max=desired:** "2 dedicated + 3 spot" was a hard capacity requirement (5 nodes total), not an autoscaling target — there is no scale-out path for these groups by design.

4.3 Security Groups

Cluster and node security groups reference each other's IDs bidirectionally (cluster needs ingress from node SG; node needs ingress from cluster SG). All rules are managed as standalone `aws_vpc_security_group_ingress_rule` / `_egress_rule` resources rather than inline `ingress` / `egress` blocks on `aws_security_group`.

> **Why standalone rule resources, not inline blocks:** two security groups that inline-reference each other's IDs create a dependency cycle Terraform cannot resolve. Standalone rule resources can reference both group IDs without the groups themselves depending on each other. Mixing inline blocks and standalone rules on the same group is also explicitly discouraged by AWS — it causes perpetual diffs as each method fights to "own" the rule set.

4.4 IAM Roles

Role	Trust principal	Purpose
cluster	eks.amazonaws.com	EKS control plane
node	ec2.amazonaws.com	Worker node instances
vpc_cni	pods.eks.amazonaws.com	VPC CNI via Pod Identity (not IRSA)
external_secrets	pods.eks.amazonaws.com	External Secrets Operator via Pod Identity
ack_capability	capabilities.eks.amazonaws.com	What the ACK controller calls AWS APIs as — the identity actually behind every `iam.services.k8s.aws` / `s3...` /etc. custom resource it provisions (detail: §8.1, §8.3)
kro_capability	capabilities.eks.amazonaws.com	Authenticates kro to AWS, but kro's actual work is Kubernetes-API orchestration, not AWS API calls — see §8.2 for why this role needs almost no AWS IAM permissions at all, unlike the other two
argocd_capability	capabilities.eks.amazonaws.com	ArgoCD EKS Capability

> **Don't conflate these capability roles with IAM resources ACK creates.** These three rows are credentials the platform's controllers use to call AWS; they are unrelated to whatever IAM Roles/Policies an ACK custom resource declared in Git ends up provisioning as a side effect. §8.3 and §8.4 walk through the distinction and how the two interact.

> **Trust policy requirement specific to capability roles:** EKS validates that a capability role's trust policy includes *both* `sts:AssumeRole` *and* `sts:TagSession`, granted to `capabilities.eks.amazonaws.com` — not the generic `eks.amazonaws.com` principal used by the cluster role. Using the wrong principal produces `CreateCapability` 400 errors with no other indication of the cause.

> **IAM propagation delay:** immediately after creating/updating a capability role's trust policy, `CreateCapability` can fail with "trust policy is invalid" even though the policy content is correct — it is racing IAM's own eventual consistency. A `time_sleep` resource (20s, keyed off the trust policy content so future edits re-wait) sits between each capability role and its capability resource.

4.5 EKS Add-ons

Add-on	Notes
vpc-cni	Pod Identity association to `kube-system/aws-node`, not IRSA
coredns	standard

kube-proxy	standard
eks-pod-identity-agent	required infrastructure — without this addon, no Pod Identity association of any kind functions, for any workload

> **`resolve_conflicts` is not a real argument** on `aws_eks_addon` — the correct arguments are `resolve_conflicts_on_create` and `resolve_conflicts_on_update`.

4.6 EKS Capabilities

EKS Capabilities are AWS's fully-managed alternative to self-hosting ACK, kro, or ArgoCD via Helm. AWS runs the controller in a service-owned account, not as pods inside the customer's cluster — eliminating install, upgrade, and scaling overhead for these three tools entirely.

```
resource "aws_eks_capability" "ack" {
  cluster_name              = aws_eks_cluster.main.name
  capability_name           = "ack"
  type                      = "ACK"
  role_arn                  = aws_iam_role.ack_capability[0].arn
  delete_propagation_policy = "RETAIN"
  depends_on                = [aws_eks_cluster.main, time_sleep.ack_capability_role]
}
```

Required arguments often missed: `type` (ACK | KRO | ARGOCD) and `delete_propagation_policy` (currently only `RETAIN` is valid).

5. Identity, Access & RBAC

5.1 Pod Identity, Not IRSA

Every workload-facing IAM role in this platform (VPC CNI, External Secrets Operator) uses **EKS Pod Identity** — trust policy to `pods.eks.amazonaws.com` plus an `aws_eks_pod_identity_association` binding a namespace/service-account pair to the role. This is a deliberate, consistent choice over the older IRSA (IAM Roles for Service Accounts / OIDC federation) pattern: Pod Identity needs no OIDC provider plumbing and is reusable across clusters.

5.2 IAM Identity Center Integration (ArgoCD SSO)

```
data "aws_ssoadmin_instances" "this" {}     # exactly one per org; resolved automatically

resource "aws_identitystore_group" "argocd_admin" {
  identity_store_id = tolist(data.aws_ssoadmin_instances.this[0].identity_store_ids)[0]
  display_name      = "ARGOCD_ADMIN"
}

# Inside the ArgoCD capability's configuration.argo_cd block:
aws_idc { idc_instance_arn = tolist(data.aws_ssoadmin_instances.this[0].arns)[0] }
rbac_role_mapping {
  role     = "ADMIN"
  identity { id = aws_identitystore_group.argocd_admin[0].group_id; type = "SSO_GROUP" }
}
```

Terraform creates the `ARGOCD_ADMIN` Identity Center group and maps it to ArgoCD's built-in `ADMIN` role. **Group membership is not Terraform-managed** — add/remove individual users via the Identity Center console as personnel changes, without touching infrastructure code.

> **Public by design:** the ArgoCD capability's server endpoint is reachable over the public internet by default. The field that would restrict it to VPC-only access via PrivateLink (`network_access.vpce_ids`) is deliberately left unset; access is gated entirely by Identity Center SSO instead.

5.3 EKS Access Entries — the Subtle Parts

When a capability is created, EKS *automatically* creates an EKS Access Entry for its IAM role on that cluster — but grants it **zero Kubernetes RBAC by default** (least privilege). Three real issues were found and fixed while wiring this up:

5.3.1 Don't create a duplicate access entry

Writing an explicit `aws_eks_access_entry` for a capability role that EKS already auto-provisioned produces `ResourceInUseException` on create. Two valid resolutions:

- **Import** the existing entry: `terraform import 'aws_eks_access_entry.x[0]' cluster_name:role_arn`
- **Delete and recreate**: `aws eks delete-access-entry ...` then `terraform apply` — acceptable since access entries hold no data, only a brief auth gap for that principal between delete and the next apply.

5.3.2 The "eks-access-entry:<arn>" group is not automatic

AWS documentation describes a group naming convention, `eks-access-entry:<principal-arn>`, that reads as if every access entry automatically belongs to such a group. It does not — that string is only meaningful if you *explicitly* set `kubernetes_groups` on the access entry to that value (or any value of your choosing). An auto-created entry has an empty group list until you set one.

5.3.3 `subject.namespace` on a Group/User RBAC subject can invalidate the whole binding

> This was the root cause of a multi-hour debugging cycle. A `kubernetes_cluster_role_binding_v1` subject block left `namespace` unset; the Terraform provider defaulted it to `"default"`. Per the Kubernetes API spec for `rbac.authorization.k8s.io/v1.Subject`: *"Namespace of the referenced object... If the object kind is non-namespace, such as User or Group, and this value is not empty the Authorizer should report an error."* The symptom was uniform "forbidden" errors across every resource type and API group, regardless of how broad the bound ClusterRole's rules were — because the binding itself was invalid, not under-scoped. Fix: explicitly set `namespace = ""` on any Group/User subject.

5.3.4 Two valid mechanisms — know which one you're using

Mechanism	Depends on Kubernetes groups?	Used for
`aws_eks_access_policy_association` (AWS-managed policy, e.g. AmazonEKSEditPolicy)	No — applies directly to the principal ARN	kro's elevated permissions; ArgoCD's namespace-scoped write
Custom `kubernetes_cluster_role_v1` + `_binding_v1`	Yes — requires the access entry's `kubernetes_groups` to be set correctly	ArgoCD's cluster-wide read (no AWS-managed policy grants true wildcard `*` / `*` view)

Where possible, prefer access policy associations — they have no group-binding failure mode at all.

6. GitOps Configuration (ArgoCD)

6.1 Self-Management — Two Independent Applications

The platform deliberately registers **two separate, independent** `Application` resources rather than nesting one inside the other:

Application	Source repo	Destination
app-of-apps	github.com/CloudScope/argo-app-of-apps	name=local-cluster, ns=argocd
argocd-self-managed	github.com/CloudScope/cloudscope-argocd-self-managed	name=local-cluster, ns=argocd

> **The destination `server` field must be the EKS cluster ARN, never the vanilla `https://kubernetes.default.svc`** that almost every ArgoCD example online uses. Per AWS's documentation for this capability: *"The default kubernetes.default.svc is not supported."* Using it would have silently prevented every Application from ever syncing, with or without a registered cluster. The destination is referenced by registered `name` (`local-cluster`) instead, which is also more stable than hardcoding an ARN twice.

6.2 Cluster Registration

The ArgoCD capability does **not** auto-register the cluster it runs on — at least one cluster must be explicitly registered before any Application can sync.

```
resource "kubernetes_secret_v1" "argocd_local_cluster" {
  metadata {
    name      = "local-cluster"
    namespace = "argocd"
    labels    = { "argocd.argoproj.io/secret-type" = "cluster" }
  }
  data = {
    name    = "local-cluster"
    server  = aws_eks_cluster.main.arn        # ARN, not a URL
    project = "default"
  }
}
```

> This secret holds **no live credentials** — just a name, the cluster ARN, and a project. Actual authorization flows through the IAM access entry the capability already has. That is what makes it safe to manage declaratively in Terraform, unlike the repository credentials below.

6.3 Private Repository Access

Both Git repositories are private and live under the same GitHub organization. Credentials are sourced from AWS Secrets Manager and applied via a single **repo-creds template** (org-wide), not a per-repository secret — any future repository under `github.com/CloudScope` is automatically covered with zero further Terraform changes.

```
# Created once, by hand, outside Terraform — never in state, tfvars, or CI secrets:
aws secretsmanager put-secret-value \
  --secret-id "GIT_HUB_PAT" \
  --secret-string '{"username":"<github-username>","token":"<github-pat>"}'
```

```
data "aws_secretsmanager_secret" "argocd_repo" {
  name = "GIT_HUB_PAT"
}

resource "kubernetes_secret_v1" "argocd_github_org_creds" {
  metadata {
    name      = "cloudscope-org-repo-creds"
    namespace = "argocd"
    labels    = { "argocd.argoproj.io/secret-type" = "repo-creds" }   # template, not "repository"
  }
  data = {
    type      = "git"
    url       = "https://github.com/CloudScope"                      # prefix match
    secretArn = data.aws_secretsmanager_secret.argocd_repo[0].arn     # ARN only — not the credential
itself
  }
}
```

The Secrets Manager JSON keys must be exactly `username` and `token` — this is the ArgoCD capability's own hardcoded parsing of the secret payload, not a Terraform or configuration choice. Any other key naming produces silent authentication failures that surface as `revision main must be resolved`, not as an obvious "bad credentials" error.

The IAM policy granting the ArgoCD capability role read access to this one secret is scoped to the exact secret ARN (`secretsmanager:GetSecretValue`, `DescribeSecret`) — not a blanket Secrets Manager grant.

UI note: ArgoCD's Settings → Repositories page only lists explicit `repository`-type secrets, never `repo-creds` templates — so it will show "No repositories connected" even though syncing works correctly. This is cosmetic, confirmed against upstream ArgoCD documentation, and was a deliberate trade-off in favor of the org-wide template over per-repo visibility.

7. External Secrets Operator

Terraform provisions only the **prerequisites** — the IAM role, a scoped Secrets Manager read policy, and the Pod Identity association — matching the External Secrets Helm chart's default namespace/ServiceAccount (`external-secrets` / `external-secrets`). The operator itself, and any `ClusterSecretStore` / `ExternalSecret` manifests, are deployed via the GitOps repos, not Terraform.

```
locals {
  external_secrets_secret_arns = length(var.external_secrets_secret_arns) > 0 ?
var.external_secrets_secret_arns : [

"arn:aws:secretsmanager:${var.aws_region}:${data.aws_caller_identity.current.account_id}:secret:${var.c
luster_name}-*"
  ]
}
```

Defaults to a naming-convention prefix (`<cluster_name>-*`) requiring zero additional input; override with exact ARNs once real secret names are known, for tighter scoping. The action list (`GetSecretValue` , `DescribeSecret` , `ListSecretVersionIds` , `BatchGetSecretValue` , `GetResourcePolicy`) was taken directly from ESO's own documented AWS Secrets Manager provider requirements. `secretsmanager:ListSecrets` is granted separately with `Resource = "*"` because IAM does not support resource-level scoping for that particular action at all.

8. ACK & kro Permission Model

8.1 ACK — AWS IAM Permissions

ACK's capability role starts with **zero** AWS IAM permissions — it cannot create any AWS resource until explicitly granted. Current scope:

Service	Policy
S3	AmazonS3FullAccess
RDS	AmazonRDSFullAccess
SQS	AmazonSQSFullAccess
SNS	AmazonSNSFullAccess
EC2 / VPC	AmazonEC2FullAccess
CloudWatch	CloudWatchFullAccessV2 (CloudWatchFullAccess is deprecated)
Lambda	AWSLambda_FullAccess
Secrets Manager	SecretsManagerReadWrite
IAM	IAMFullAccess
EKS	Custom policy (no managed policy exists) — taken verbatim from ACK's own published `config/iam/recommended-inline-policy` for the `eks-controller` : `eks:*, iam:GetRole, iam:PassRole, iam:ListAttachedRolePolicies, ec2:DescribeSubnets`

> **Blast radius:** `IAMFullAccess` and EC2/VPC full access give the ACK controller effectively IAM-admin-equivalent reach, account-wide, through any custom resource merged into either GitOps repository. This was an explicit, requested scope — not a default — and is the single highest-value target for tightening if/when the actual set of managed services narrows.

8.2 kro — Kubernetes RBAC, Not AWS IAM

kro itself needs essentially no AWS IAM permissions — its job is orchestrating Kubernetes custom resources, not calling AWS APIs directly. What it needs is **Kubernetes RBAC broad enough to act on whatever resource kinds any `ResourceGraphDefinition` references.**

Grant	Why it wasn't enough
Auto-granted `AmazonEKSKROPolicy`	Covers only kro's own `kro.run` CRDs/RGDs — not the resources an RGD composes
`AmazonEKSEditPolicy` (added first)	A fixed, AWS-enumerated list of API groups (apps, batch, networking.k8s.io, core resources, ...) — never included ACK's custom resource groups (`iam.services.k8s.aws` , `s3.services.k8s.aws` , etc.) at all
`AmazonEKSClusterAdminPolicy` (current)	Full `*` / `*` / `*` — unblocks every current and future ACK CRD kind an RGD might reference

> **Pattern recognized twice in this build:** patching Kubernetes RBAC permission-by-permission, resource-kind-by-resource-kind, does not converge — there is always another CRD kind a new RGD or Application will reference next. Both times (ArgoCD's read access, kro's write access) the fix was a single blanket grant rather than incremental ones. `aws_eks_access_policy_association` is the lower-risk way to do this where an AWS-managed policy fits, since it applies directly to the principal with no Kubernetes group/binding mechanism in the loop at all.

8.3 Two Different Things Both Called "IAM" Here — Don't Conflate Them

This platform involves IAM in two unrelated senses, and the permission model only makes sense once they're kept separate:

	What it is	Example in this platform
IAM roles *this platform uses*	Credentials the EKS Capabilities (and other controllers) authenticate to AWS with — i.e. *who is calling AWS APIs*	`ack_capability`, `kro_capability`, `argocd_capability` roles (§4.4)
IAM resources *ACK provisions*	Real AWS IAM Roles/Policies that exist because someone declared an `iam.services.k8s.aws/Role` or `/Policy` custom resource in Git — i.e. *what gets created as a side effect*	e.g. an application's own IAM role for Pod Identity, provisioned declaratively instead of by hand

The `IAMFullAccess` policy in §8.1 belongs entirely to the first category — it is what lets the `ack_capability` role's controller *call* IAM's `CreateRole` / `CreatePolicy` /etc. APIs on behalf of whatever `iam.services.k8s.aws` custom resources show up in either GitOps repo. It has nothing to do with the permissions of the roles ACK ends up creating — those are whatever the custom resource's own spec declares.

8.4 How kro, ACK, and AWS IAM Actually Relate

A concrete walk-through — kro composing an IAM role (the platform's `iamrolewithsecret-rgd.yaml` pattern) for an application that needs to read a secret:

```
1. A developer creates an INSTANCE of an existing ResourceGraphDefinition in Git
   (e.g. "give my app an IAM role that can read secret X").
        |
        ▼
2. ArgoCD syncs it — the instance is just a small Kubernetes custom resource.
   (ArgoCD's own RBAC, §5, is what lets it write this object to the cluster.)
        |
        ▼
3. kro reads the RGD + instance and expands it into the underlying resources the
   RGD's "resources" list declares — here, an ACK "iam.services.k8s.aws/Role" object
   (and usually a matching "/Policy" and an eks_pod_identity_association elsewhere).
   kro is only creating KUBERNETES OBJECTS at this point — no AWS API call has
   happened yet. This is why kro itself needs no AWS IAM permissions (§8.2) — it
   only needs Kubernetes RBAC broad enough to create those custom resource objects.
        |
        ▼
4. The ACK capability's IAM controller is watching iam.services.k8s.aws resources
   cluster-wide. It notices the new Role object and calls the REAL AWS IAM API
   (iam:CreateRole) to make it actually exist — authenticating as the
   ack_capability role, using the IAMFullAccess policy from §8.1.
        |
        ▼
```

```
5. ACK writes the real role's ARN/status back onto the Kubernetes object, which
   kro and any dependent resources in the same RGD (e.g. a Pod Identity
   association referencing that role) can then read.
```

The takeaway: kro's permissions control whether the *Kubernetes representation* of a resource can be created; ACK's permissions control whether the *real AWS resource* behind it can be created. They are two separate authorization checks in the same pipeline, against two different APIs (the Kubernetes API server, and the AWS IAM API), using two different IAM roles. Granting kro more Kubernetes RBAC does nothing for ACK's AWS-side permissions, and vice versa — both had to be fixed independently in this build (§8.1, §8.2), and a gap in either one stops the same end-to-end flow.

Why this makes `ack_capability`'s `IAMFullAccess` the platform's single biggest privilege concentration: any RGD or Application merged into either GitOps repo that declares an `iam.services.k8s.aws` resource is, transitively, exercising `IAMFullAccess` on the real AWS account. kro's broad Kubernetes RBAC (§8.2) is what lets that declaration reach ACK's controller in the first place. Tightening either side alone is not enough — both the Kubernetes-side reach (kro) and the AWS-side reach (ACK) would need narrowing together to meaningfully reduce this blast radius (see §12).

9. CI/CD Pipeline

GitHub Actions (`.github/workflows/terraform-deploy.yml`):

Trigger	Action
Pull request	`fmt -check` , `init` , `validate` , `plan` , plan posted as a PR comment
Push to main/master	All of the above, plus `apply -auto-approve` against the saved plan, then `terraform output -json` uploaded as a build artifact

Secrets handling: `terraform.tfvars` is gitignored and therefore never reaches CI. Any variable that must be supplied to GitHub Actions specifically is wired as `TF_VAR_*` at the job level, sourced from GitHub Actions secrets — never committed, never duplicated into the Terraform-managed Secrets Manager flow.

Authentication consistency matters: EKS automatically grants the IAM principal that created the cluster implicit admin-equivalent Kubernetes access. Resources using the `kubernetes` provider (the ArgoCD `Application` /RBAC objects) depend on this holding true — i.e. the same GitHub Actions role must be the one that has run every apply against this cluster. A different applying principal will need an explicit `aws_eks_access_entry` of its own first.

10. Step-by-Step Implementation Procedure

1. **Confirm prerequisites** — existing VPC, IAM Identity Center enabled, GitHub org with the two target repos created (with at least one commit / a `main` branch), AWS provider pinned >= 6.25.0.
2. **Create the GitHub PAT** with read access scoped to every current and future repo it needs (an org-wide fine-grained token avoids the per-repo access gotcha documented in §11).
3. **Store the PAT in Secrets Manager** as `GIT_HUB_PAT` with JSON keys exactly `username` / `token` — before the first apply that references it.
4. **Apply the network/cluster/node-group layer** (`main.tf` , `iam.tf` , `security_groups.tf` , `eks_cluster.tf` , `node_groups.tf`) to bring up the EKS cluster and Graviton node groups.
5. **Apply the capabilities layer** (`capabilities.tf`) — ACK, kro, and ArgoCD capability resources, their IAM roles and trust policies, and the `time_sleep` guards for IAM propagation.
6. **Apply the GitOps bootstrap** (`argocd_apps.tf`) — repo-creds template, local-cluster registration, RBAC for the ArgoCD capability role, and the two root `Application` resources.
7. **Apply remaining prerequisites** (`external_secrets.tf`) for ESO's IAM/Pod Identity wiring.
8. **Verify capability health** — confirm all three capabilities show `ACTIVE` in the EKS console, and that the ArgoCD server URL (`terraform output argocd_url`) resolves.
9. **Add ArgoCD admin users** — Identity Center console → Groups → `ARGOCD_ADMIN` → add the people who need admin access (this is intentionally not Terraform-managed).
10. **Confirm GitOps sync** — both Applications should reach `Synced` / `Healthy` . Validate end-to-end by committing one trivial ACK custom resource (e.g. an S3 bucket CR) to a GitOps repo and confirming the real AWS resource is created.
11. **Build out the application layer** — kro `ResourceGraphDefinition` s composing ACK resources into the higher-level, self-service APIs application teams actually need.

11. Troubleshooting & Lessons Learned

This section documents real issues encountered during the build, in case they recur.

Symptom	Root Cause	Fix
EKS addon apply errors on `resolve_conflicts`	Not a real argument on `aws_eks_addon`	Use `resolve_conflicts_on_create` / `_on_update`
Terraform error referencing `aws_autoscaling_groups_data`	Hallucinated/nonexistent data source name	Real name is `aws_autoscaling_groups` (no `_data` suffix)
`vpc_config` rejected on `aws_eks_node_group`	That argument does not exist on this resource at all	Attach a custom SG via `launch_template` instead
`taints` block rejected	Block name is singular	Use `taint`
`CreateCapability` 400: trust policy invalid	Wrong principal (`eks.amazonaws.com`) and/or missing `sts:TagSession`	Principal must be `capabilities.eks.amazonaws.com` with both actions; add a `time_sleep` for propagation
`CreateAccessEntry` 409: ResourceInUseException	EKS already auto-created the entry as a side effect of capability/node-group creation	Import the existing resource, or delete it on the AWS side and let Terraform recreate cleanly
EC2 node group creation: `Ec2InstanceTypeDoesNotExist`	Instance type not offered in the target region	Verify with `aws ec2 describe-instance-type-offerings` before adding new types
`kubernetes_manifest` : "Unauthorized" from the API server	Statically-fetched `data.aws_eks_cluster_auth` token (~15 min validity) expired before a long apply reached that resource	Switch the `kubernetes` provider to `exec` -based auth (`aws eks get-token`), generating a fresh token per API call
ArgoCD Application never syncs, no cluster errors visible	`destination.server = "https://kubernetes.default.svc"` is unsupported by this capability	Register the cluster by ARN and reference it via `destination.name`
Uniform "forbidden" on every resource kind regardless of RBAC rule breadth	`ClusterRoleBinding` subject had a stray `namespace` field on a Group-kind subject, invalidating the entire binding per the Kubernetes API spec	Explicitly set `namespace = ""` on Group/User subjects
`kubernetes_groups[0]` : "Invalid index"	`kubernetes_groups` on `aws_eks_access_entry` is a set, not a list — sets have no index	Define the group name once as a `local` and reuse it, instead of reading it back off the resource
`ArgoCD: "revision main must be resolved"`	Either (a) the PAT's fine-grained repository access excludes that repo, (b) the Secrets Manager JSON used the wrong key names, or (c) the token was rotated/expired	Verify PAT repo access first (does not require a Terraform change); confirm JSON keys are exactly `username` / `token`
ArgoCD Settings → Repositories shows "No repositories connected" despite working syncs	The page only lists explicit `repository` -type secrets, never `repo-creds` templates	Cosmetic; add explicit per-repo secrets only if UI visibility is required, at the cost of losing automatic coverage for future repos
kro can create RGDs but not what they compose (e.g. ACK `Policy` / `Role`)	`AmazonEKSEditPolicy` never covered ACK's custom resource API groups	Escalate to `AmazonEKSClusterAdminPolicy` via `aws_eks_access_policy_association` (no group-binding risk, unlike custom ClusterRoles)

`terraform init`: "Duplicate required providers configuration"	A Terraform module may have only one `required_providers` block, even split across files	Consolidate all provider declarations into a single block

12. Security Considerations & Follow-ups

- **ACK's `IAMFullAccess` grant** is the single largest blast-radius item in this platform — any reviewer with merge access to either GitOps repo can, in effect, create or escalate IAM roles. Recommended follow-up: narrow to a custom policy scoped by role-name prefix once the actual set of IAM resources ACK needs to manage is known.
- **kro, ACK's own AWS IAM scope, and the ArgoCD capability all currently hold full `AmazonEKSClusterAdminPolicy`** (the latter two for the first-time-apply safety reasons in §11; kro per §8.2's whack-a-mole finding). These were chosen deliberately over continued incremental, per-resource-kind patching. **Narrowing kro's Kubernetes RBAC alone would not reduce the IAM blast radius described in §8.4** — ACK's own AWS-side `IAMFullAccess` would still allow the same end result via any other path that reaches an `iam.services.k8s.aws` custom resource. The two need to be narrowed together, not independently, to be meaningful. A custom `ClusterRole` scoped to exactly the API groups in active use is the tighter Kubernetes RBAC alternative (apply the namespace-subject lesson from §5.3.3 if doing so).
- **ArgoCD's server is public by design**, gated by Identity Center SSO rather than network controls. Re-evaluate if a private-only posture becomes a compliance requirement (`network_access.vpce_ids`).
- **The current applying principal's implicit cluster-admin access** (granted by EKS to whoever created the cluster) is a single point of coupling between the GitHub Actions role and several Terraform-managed Kubernetes resources. Document or pin that role explicitly rather than relying on "whoever happened to run the first apply."

13. Appendix: File & Variable Reference

Terraform files (`terraform/`)

File	Contents
main.tf	Providers, VPC/subnet/route-table resources
variables.tf / terraform.tfvars	All input variables and their values
iam.tf	Cluster and node IAM roles
security_groups.tf	Cluster/node security groups and standalone rules
eks_cluster.tf	EKS cluster resource, Pod Identity addon, VPC CNI, CoreDNS, kube-proxy
node_groups.tf	Launch template, on-demand and spot managed node groups, ASG tags
capabilities.tf	ACK / kro / ArgoCD capability resources, their IAM roles, RBAC/access-policy associations, Identity Center integration
argocd_apps.tf	Kubernetes provider, repo-creds template, cluster registration, ArgoCD capability RBAC, the two root Applications
external_secrets.tf	External Secrets Operator IAM role, scoped Secrets Manager policy, Pod Identity association
outputs.tf	All output values, including `argocd_url` and capability role ARNs
backend.tf	S3 remote state backend with native S3 locking

Key variables

Variable	Default	Purpose
cluster_version	1.36	Kubernetes version
on_demand_desired_size / spot_desired_size	2 / 3	Fixed node counts
on_demand_instance_types / spot_instance_types	t4g.large, m6g.large	Graviton instance families, confirmed available in ap-south-1
node_ami_type	AL2023_ARM_64_STANDARD	Must match the arm64 instance types above
argocd_admin_group_name	ARGOCD_ADMIN	Identity Center group mapped to ArgoCD's ADMIN role
argocd_apps_repo_url / argocd_self_managed_repo_url	the two CloudScope repos	GitOps sources
external_secrets_secret_arns	[] (falls back to a naming-convention prefix)	Override with exact ARNs once known